

Problema 02: Análise de Estabilidade e Custo em Sistemas de Abastecimento de Água

Prompt Utilizado

Desenvolver um programa que analise o desempenho do sistema de abastecimento para N bairros, onde N é definido pelo usuário a cada execução. O programa deve calcular indicadores de eficiência, risco de desabastecimento e custo operacional, além de gerar gráficos comparativos que permitam visualizar rapidamente a situação de cada bairro e do sistema como um todo.

A solução deve ser implementada em Python, utilizando vetores/listas de tamanho variável, definidos a cada execução. Os resultados devem ser apresentados em forma de tabela e também por meio de gráficos visuais.

A cada execução, o programa deverá solicitar:

- número de bairros N
- nome de cada bairro
- demanda diária estimada, em m³
- capacidade do reservatório, em m³
- volume inicial disponível em cada reservatório, em m³
- percentual de perda na tubulação
- custo de bombeamento por m³

Chat-GPT:

```
import matplotlib.pyplot as plt
```

```
# =====
```

```
# ENTRADA DE DADOS
```

```
# =====
```

```
N = int(input("Digite o número de bairros: "))
```

```
bairros = []

demanda = []

capacidade = []

volume_inicial = []

perda = []

custo_bombeamento = []


for i in range(N):

    print(f"\n--- Bairro {i+1} ---")


    bairros.append(input("Nome do bairro: "))


    demanda.append(float(input("Demanda diária estimada (m³): ")))


    capacidade.append(float(input("Capacidade do reservatório (m³): ")))


    volume_inicial.append(float(input("Volume inicial disponível (m³): ")))


    perda.append(float(input("Percentual de perda na tubulação (%): ")))


    custo_bombeamento.append(float(input("Custo de bombeamento por m³ (R$): ")))
```

```
# =====

# PROCESSAMENTO

# =====

eficiencia = []

risco = []

custo_total = []

volume_util = []

for i in range(N):

    # Volume realmente disponível após perdas

    volume_disponivel = volume_inicial[i] * (1 - perda[i] / 100)

    volume_util.append(volume_disponivel)

    # Eficiência (%)

    ef = (volume_disponivel / demanda[i]) * 100

    eficiencia.append(ef)

    # Risco de desabastecimento

    if volume_disponivel >= demanda[i]:

        risco.append("BAIXO")

    elif volume_disponivel >= 0.7 * demanda[i]:

        risco.append("MÉDIO")
```

```

else:

    risco.append("ALTO")

# Custo operacional

custo = volume_disponivel * custo_bombeamento[i]

custo_total.append(custo)

# =====

# SAÍDA EM TABELA

# =====

print("\n===== RESULTADOS =====\n")

print(f"{'Bairro':15} {'Eficiência (%)':15} {'Risco':10} {'Custo (R$) ':15}")

for i in range(N):

    print(f"{'bairros[i]:15} "

          f"{'eficiencia[i]:15.2f} "

          f"{'risco[i]:10} "

          f"{'custo_total[i]:15.2f}")

# =====

# GRÁFICOS

# =====

```

```
plt.figure(figsize=(14, 6))

# -----

# Gráfico de eficiência

# -----

plt.subplot(1, 2, 1)

plt.bar(bairros, eficiencia, color='skyblue')

plt.axhline(100, color='green', linestyle='--', label='Meta ideal')

plt.title("Eficiência do Sistema")

plt.ylabel("Eficiência (%)")

plt.xlabel("Bairros")

plt.legend()

# -----

# Gráfico de custos

# -----

plt.subplot(1, 2, 2)

plt.bar(bairros, custo_total, color='orange')

plt.title("Custo Operacional")
```

```
plt.ylabel("Custo (R$)")

plt.xlabel("Bairros")


plt.tight_layout()

plt.show()
```

Gemini:

```
import matplotlib.pyplot as plt


def analisar_abastecimento():

    print("--- Sistema de Análise de Abastecimento de Água ---")


    # 1. Entrada do número de bairros

    try:

        n = int(input("Informe o número de bairros (N): "))

    except ValueError:

        print("Erro: O número de bairros deve ser um inteiro.")

        return


    # Inicialização das listas (vetores de tamanho variável)

    nomes = []

    demandas = []

    capacidades = []

    volumes_iniciais = []

    perdas_perc = []
```

```
custos_unitarios = []
```

```
# Listas para resultados calculados
```

```
eficiencias = []
```

```
riscos = []
```

```
custos_totais = []
```

```
# 2. Coleta de dados por bairro
```

```
for i in range(n):
```

```
    print(f"\nDados para o bairro {i+1}:")
```

```
    nomes.append(input("Nome do bairro: "))
```

```
    demandas.append(float(input("Demanda diária estimada (m³): ")))
```

```
    capacidades.append(float(input("Capacidade do reservatório (m³): ")))
```

```
    volumes_iniciais.append(float(input("Volume inicial disponível (m³): ")))
```

```
    perdas_perc.append(float(input("Percentual de perda na tubulação (%): ")))
```

```
    custos_unitarios.append(float(input("Custo de bombeamento por m³ (R$): ")))
```

```
# 3. Processamento dos Indicadores
```

```
for i in range(n):
```

```
    # Perda em volume: (Demanda * Perda%) / 100
```

```
    volume_perda = (demandas[i] * perdas_perc[i]) / 100
```

```
    volume_necessario = demandas[i] + volume_perda
```

```
    # Eficiência: (Volume Inicial / Demanda) * 100 (capacidade de suprir a demanda  
    imediata)
```

```
    eficiencia = (volumes_iniciais[i] / volume_necessario) * 100 if volume_necessario > 0  
    else 0
```

```
eficiencias.append(round(eficiencia, 2))
```

```
# Risco de Desabastecimento: Se a demanda + perda for maior que a capacidade total
```

```
risco = (volume_necessario / capacidades[i]) * 100
```

```
riscos.append(round(risco, 2))
```

```
# Custo Operacional: Volume necessário * custo de bombeamento
```

```
custo = volume_necessario * custos_unitarios[i]
```

```
custos_totais.append(round(custo, 2))
```

```
# 4. Apresentação dos Resultados em Tabela
```

```
print("\n" + "="*85)
```

```
print(f'{"Bairro":<15} | {"Eficiência (%)":<15} | {"Risco (%)":<12} | {"Custo Total (R$)":<15}')
```

```
print("-" * 85)
```

```
for i in range(n):
```

```
    print(f'{"nomes[i]":<15} | {"eficiencias[i]":<15} | {"riscos[i]":<12} | {"custos_totais[i]":<15}')
```

```
print("="*85)
```

```
# 5. Geração de Gráficos Comparativos
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

```
# Gráfico 1: Eficiência e Risco
```

```
x = range(len(nomes))
```

```
ax1.bar(x, eficiencias, width=0.4, label='Eficiência (%)', align='center', color='skyblue')
```

```
ax1.bar(x, riscos, width=0.4, label='Risco (%)', align='edge', color='salmon')
```

```
ax1.set_xticks(x)
```



```

ax1.set_xticklabels(nomes)

ax1.set_title('Eficiência vs Risco por Bairro')

ax1.legend()


# Gráfico 2: Custo Operacional

ax2.pie(custos_totais, labels=nomes, autopct='%1.1f%%', startangle=140,
colors=plt.cm.Paired.colors)

ax2.set_title('Distribuição de Custos Operacionais')


plt.tight_layout()

plt.show()


# Executar o programa

if __name__ == "__main__":

    analisar_abastecimento()

```

Claude:

```

"""
Sistema de Análise de Desempenho do Abastecimento de Água
=====
Analisa N bairros com indicadores de eficiência, risco de
desabastecimento e custo operacional, gerando tabelas e gráficos.
"""

import os
import sys

try:
    import matplotlib.pyplot as plt
    import matplotlib.patches as mpatches
    import matplotlib.gridspec as gridspec
    from matplotlib.colors import LinearSegmentedColormap
    import numpy as np
except ImportError:

```

```
print("Instalando dependências necessárias...")
os.system(f"{sys.executable} -m pip install matplotlib numpy --quiet")
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.gridspec as gridspec
from matplotlib.colors import LinearSegmentedColormap
import numpy as np
```

```
# -----
# FUNÇÕES DE CÁLCULO
# -----
```

```
def calcular_volume_util(volume_inicial: float, perda_pct: float) -> float:
    """Volume disponível após desconto das perdas na tubulação."""
    return volume_inicial * (1 - perda_pct / 100)
```

```
def calcular_eficiencia(volume_util: float, demanda: float) -> float:
    """Eficiência de atendimento (%) — máximo 100%."""
    if demanda <= 0:
        return 100.0
    return min(volume_util / demanda * 100, 100.0)
```

```
def calcular_risco(volume_util: float, demanda: float, capacidade: float) -> str:
    """
    Classifica o risco de desabastecimento:
    CRÍTICO — volume útil < 50 % da demanda
    ALTO — volume útil entre 50 % e 80 % da demanda
    MODERADO — volume útil entre 80 % e 100 % da demanda
    BAIXO — volume útil >= demanda E reservatório >= 70 % cheio
    NORMAL — demais casos
    """
    if demanda <= 0:
        return "BAIXO"
    cobertura = volume_util / demanda
    nivel_reservatorio = volume_util / capacidade if capacidade > 0 else 0

    if cobertura < 0.50:
        return "CRÍTICO"
    elif cobertura < 0.80:
        return "ALTO"
    elif cobertura < 1.00:
        return "MODERADO"
    elif nivel_reservatorio >= 0.70:
        return "BAIXO"
    else:
```

```
return "NORMAL"
```

```
def calcular_custo(demanda: float, perda_pct: float, custo_por_m3: float) -> float:  
    """
```

```
    Custo operacional diário de bombeamento.  
    É necessário bombear mais água para compensar as perdas.  
    """
```

```
    fator_perda = 1 / (1 - perda_pct / 100) if perda_pct < 100 else 1  
    return demanda * fator_perda * custo_por_m3
```

```
def calcular_autonomia(volume_util: float, demanda: float) -> float:
```

```
    """Dias de autonomia com o volume disponível."""
```

```
    if demanda <= 0:  
        return float("inf")  
    return volume_util / demanda
```

```
# _____  
# ENTRADA DE DADOS  
# _____
```

```
def ler_float(prompt: str, minimo: float = 0.0, maximo: float = float("inf")) -> float:  
    while True:
```

```
        try:  
            valor = float(input(prompt).replace(",", "."))  
            if minimo <= valor <= maximo:  
                return valor  
            print(f" ⚠ Digite um valor entre {minimo} e {maximo}.")  
        except ValueError:  
            print(" ⚠ Entrada inválida. Use apenas números.")
```

```
def ler_int(prompt: str, minimo: int = 1) -> int:
```

```
    while True:  
        try:  
            valor = int(input(prompt))  
            if valor >= minimo:  
                return valor  
            print(f" ⚠ Digite um inteiro >= {minimo}.")  
        except ValueError:  
            print(" ⚠ Entrada inválida.")
```

```
def coletar_dados() -> list[dict]:
```

```
    print("\n" + "=" * 62)  
    print(" SISTEMA DE ANÁLISE DE ABASTECIMENTO DE ÁGUA")
```

```

print("=" * 62)

n = ler_int("\nNúmero de bairros (N): ", minimo=1)
bairros = []

for i in range(n):
    print(f"\n—— Bairro {i + 1} de {n} " + "-" * 30)
    nome = input(" Nome do bairro: ").strip() or f"Bairro {i + 1}"

    demanda = ler_float(" Demanda diária estimada (m³): ", 0.1)
    capacidade = ler_float(" Capacidade do reservatório (m³): ", 0.1)
    volume_ini = ler_float(
        f" Volume inicial disponível (m³) [0 – {capacidade}]: ",
        0.0, capacidade
    )
    perda = ler_float(" Percentual de perda na tubulação (%): ", 0.0, 99.9)
    custo = ler_float(" Custo de bombeamento por m³ (R$): ", 0.0)

    # Cálculos derivados
    vol_util = calcular_volume_util(volume_ini, perda)
    eficiencia = calcular_eficiencia(vol_util, demanda)
    risco = calcular_risco(vol_util, demanda, capacidade)
    custo_op = calcular_custo(demanda, perda, custo)
    autonomia = calcular_autonomia(vol_util, demanda)

    bairros.append({
        "nome": nome,
        "demanda": demanda,
        "capacidade": capacidade,
        "vol_inicial": volume_ini,
        "perda": perda,
        "custo_m3": custo,
        "vol_util": vol_util,
        "eficiencia": eficiencia,
        "risco": risco,
        "custo_op": custo_op,
        "autonomia": autonomia,
    })

return bairros

# —————
# TABELA DE RESULTADOS
# —————

CORES_RISCO = {
    "CRÍTICO": "\033[91m", # vermelho

```

```

"ALTO": "\033[93m", # amarelo
"MODERADO": "\033[33m", # laranja (amarelo escuro)
"NORMAL": "\033[92m", # verde
"BAIXO": "\033[96m", # ciano
}
RESET = "\033[0m"

```

```

def imprimir_tabela(bairros: list[dict]) -> None:
    colunas = [
        ("Bairro", "nome", 20, "s"),
        ("Demanda\n(m³)", "demanda", 10, ".1f"),
        ("Cap.\n(m³)", "capacidade", 10, ".1f"),
        ("Vol.Útil\n(m³)", "vol_util", 10, ".1f"),
        ("Efic.\n(%)", "eficiencia", 8, ".1f"),
        ("Perda\n(%)", "perda", 8, ".1f"),
        ("Custo Op.\n(R$)", "custo_op", 12, ".2f"),
        ("Autonomia\n(dias)", "autonomia", 11, ".2f"),
        ("Risco", "risco", 10, "s"),
    ]

    print("\n" + "=" * 62)
    print(" RESULTADOS POR BAIRRO")
    print("=" * 62)

    # Cabeçalho
    header = ""
    for titulo, _, largura, _ in colunas:
        linha1 = titulo.split("\n")[0]
        header += linha1.center(largura + 2)
    print(header)
    header2 = ""
    for titulo, _, largura, _ in colunas:
        partes = titulo.split("\n")
        linha2 = partes[1] if len(partes) > 1 else ""
        header2 += linha2.center(largura + 2)
    print(header2)
    print("-" * (sum(l + 2 for _, _, l, _ in colunas)))

    for b in bairros:
        cor = CORES_RISCO.get(b["risco"], "")
        linha = ""
        for _, chave, largura, fmt in colunas:
            val = b[chave]
            if fmt == "s":
                cel = str(val).ljust(largura)
            elif "f" in fmt:
                if b[chave] == float("inf"):

```

```

        cel = "∞".rjust(largura)
    else:
        cel = format(val, fmt).rjust(largura)
    else:
        cel = str(val).rjust(largura)
    linha += " " + cel
    print(f"{cor}{linha}{RESET}")

print("-" * (sum(l + 2 for _, _, l, _ in colunas)))

# Totais / médias
total_demanda = sum(b["demanda"] for b in bairros)
total_vol_util = sum(b["vol_util"] for b in bairros)
total_custo = sum(b["custo_op"] for b in bairros)
media_efic = np.mean([b["eficiencia"] for b in bairros])
media_perda = np.mean([b["perda"] for b in bairros])

print(f"\n {'TOTAIS / MÉDIAS DO SISTEMA':30s}")
print(f" Demanda total diária : {total_demanda:>10.1f} m³")
print(f" Volume útil total : {total_vol_util:>10.1f} m³")
print(f" Custo operacional/dia : R$ {total_custo:>9.2f}")
print(f" Eficiência média : {media_efic:>10.1f} %")
print(f" Perda média : {media_perda:>10.1f} %")

# _____
# GRÁFICOS
# _____

PALETA_RISCO = {
    "CRÍTICO": "#e74c3c",
    "ALTO": "#e67e22",
    "MODERADO": "#f1c40f",
    "NORMAL": "#2ecc71",
    "BAIXO": "#1abc9c",
}

def gerar_graficos(bairros: list[dict], salvar: bool = True) -> None:
    nomes = [b["nome"] for b in bairros]
    n = len(bairros)
    x = np.arange(n)
    cores = [PALETA_RISCO[b["risco"]] for b in bairros]

    fig = plt.figure(figsize=(18, 14), facecolor="#0d1117")
    fig.suptitle(
        "SISTEMA DE ANÁLISE DE ABASTECIMENTO DE ÁGUA",
        fontsize=16, fontweight="bold", color="white", y=0.98
    )

```

)

```
gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.55, wspace=0.38)
```

```
# — 1. Eficiência por bairro (barras) —————
```

```
ax1 = fig.add_subplot(gs[0, :2])
barras = ax1.bar(x, [b["eficiencia"] for b in bairros],
                 color=cores, edgecolor="#ffffff22", linewidth=0.5)
ax1.axhline(100, color="#ffffff55", linestyle="--", linewidth=1)
ax1.set_title("Eficiência de Atendimento (%)", color="white", fontsize=11, pad=8)
ax1.set_xticks(x); ax1.set_xticklabels(nomes, rotation=30, ha="right", color="white",
fontsize=8)
ax1.set_ylim(0, 115)
ax1.set_facecolor("#161b22"); ax1.tick_params(colors="white")
for spine in ax1.spines.values(): spine.set_edgecolor("#30363d")
for barra, b in zip(barras, bairros):
    ax1.text(barra.get_x() + barra.get_width() / 2,
             barra.get_height() + 1.5,
             f"{b['eficiencia']:.1f}%", ha="center", va="bottom",
             color="white", fontsize=7.5, fontweight="bold")
```

```
# — 2. Pizza de risco —————
```

```
ax2 = fig.add_subplot(gs[0, 2])
contagem_risco = {}
for b in bairros:
    contagem_risco[b["risco"]] = contagem_risco.get(b["risco"], 0) + 1
labels_r = list(contagem_risco.keys())
sizes_r = list(contagem_risco.values())
cores_r = [PALETA_RISCO[r] for r in labels_r]
wedges, texts, autotexts = ax2.pie(
    sizes_r, labels=labels_r, colors=cores_r,
    autopct="%1.0f%%", startangle=140,
    textprops={"color": "white", "fontsize": 8},
    wedgeprops={"edgecolor": "#0d1117", "linewidth": 1.5}
)
for at in autotexts: at.set_fontsize(7)
ax2.set_title("Distribuição de Risco", color="white", fontsize=11, pad=8)
ax2.set_facecolor("#161b22")
```

```
# — 3. Demanda vs Volume útil (barras agrupadas) —————
```

```
ax3 = fig.add_subplot(gs[1, :2])
w = 0.38
ax3.bar(x - w/2, [b["demanda"] for b in bairros], w,
        label="Demanda (m³)", color="#3498db", alpha=0.85, edgecolor="#ffffff22")
ax3.bar(x + w/2, [b["vol_util"] for b in bairros], w,
        label="Vol. Útil (m³)", color=cores, edgecolor="#ffffff22", alpha=0.85)
ax3.set_title("Demanda x Volume Útil Disponível (m³)", color="white", fontsize=11, pad=8)
```

```

ax3.set_xticks(x); ax3.set_xticklabels(nomes, rotation=30, ha="right", color="white",
fontsize=8)
ax3.set_facecolor("#161b22"); ax3.tick_params(colors="white")
for spine in ax3.spines.values(): spine.set_edgecolor("#30363d")
leg = ax3.legend(fontsize=8, facecolor="#21262d", edgecolor="#30363d",
labelcolor="white")

```

— 4. Custo operacional por bairro —————

```

ax4 = fig.add_subplot(gs[1, 2])
ax4.barh([b["nome"] for b in bairros],
         [b["custo_op"] for b in bairros],
         color=cores, edgecolor="#ffffff22")
ax4.set_title("Custo Operacional Diário (R$)", color="white", fontsize=10, pad=8)
ax4.set_facecolor("#161b22"); ax4.tick_params(colors="white", labels=7)
for spine in ax4.spines.values(): spine.set_edgecolor("#30363d")
ax4.invert_yaxis()

```

— 5. Nível do reservatório (%) —————

```

ax5 = fig.add_subplot(gs[2, :2])
niveis = [b["vol_util"] / b["capacidade"] * 100 for b in bairros]
bars5 = ax5.bar(x, niveis, color=cores, edgecolor="#ffffff22", linewidth=0.5)
ax5.axhline(70, color="#f1c40f", linestyle="--", linewidth=1, label="70 % (referência)")
ax5.axhline(30, color="#e74c3c", linestyle=":", linewidth=1, label="30 % (alerta)")
ax5.set_title("Nível do Reservatório (% da capacidade)", color="white", fontsize=11,
pad=8)
ax5.set_xticks(x); ax5.set_xticklabels(nomes, rotation=30, ha="right", color="white",
fontsize=8)
ax5.set_ylim(0, 115)
ax5.set_facecolor("#161b22"); ax5.tick_params(colors="white")
for spine in ax5.spines.values(): spine.set_edgecolor("#30363d")
ax5.legend(fontsize=8, facecolor="#21262d", edgecolor="#30363d", labelcolor="white")
for barra, nivel in zip(bars5, niveis):
    ax5.text(barra.get_x() + barra.get_width() / 2,
             nivel + 1.5, f"{nivel:.1f}%",
             ha="center", va="bottom", color="white", fontsize=7.5)

```

— 6. Autonomia (dias) —————

```

ax6 = fig.add_subplot(gs[2, 2])
autonomias = [min(b["autonomia"], 30) for b in bairros] # cap em 30 dias p/ visualização
ax6.barh([b["nome"] for b in bairros], autonomias,
         color=cores, edgecolor="#ffffff22")
ax6.axvline(1, color="#e74c3c", linestyle="--", linewidth=1)
ax6.set_title("Autonomia (dias, cap. 30)", color="white", fontsize=10, pad=8)
ax6.set_facecolor("#161b22"); ax6.tick_params(colors="white", labels=7)
for spine in ax6.spines.values(): spine.set_edgecolor("#30363d")
ax6.invert_yaxis()

```

— Legenda de risco —————


```

patches = [mpatches.Patch(color=c, label=r) for r, c in PALETA_RISCO.items()]
fig.legend(handles=patches, loc="lower center", ncol=5, fontsize=8,
           facecolor="#21262d", edgecolor="#30363d", labelcolor="white",
           title="Nível de Risco", title_fontsize=9,
           bbox_to_anchor=(0.5, 0.01))

plt.tight_layout(rect=[0, 0.05, 1, 0.97])

if salvar:
    caminho = "relatorio_abastecimento.png"
    plt.savefig(caminho, dpi=150, bbox_inches="tight", facecolor="#0d1117")
    print(f"\n ✓ Gráfico salvo em: {caminho}")

plt.show()

```

```

# -----
# RELATÓRIO TEXTUAL DE ALERTAS
# -----

```

```

def imprimir_alertas(bairros: list[dict]) -> None:
    print("\n" + "=" * 62)
    print(" ALERTAS E RECOMENDAÇÕES")
    print("=" * 62)

    criticos = [b for b in bairros if b["risco"] == "CRÍTICO"]
    altos = [b for b in bairros if b["risco"] == "ALTO"]
    moderados = [b for b in bairros if b["risco"] == "MODERADO"]
    alta_perda = [b for b in bairros if b["perda"] > 20]

    if not (criticos or altos or moderados or alta_perda):
        print(" ✓ Todos os bairros operam em situação satisfatória.")
        return

    for b in criticos:
        print(f"\033[91m 🚨 CRÍTICO – {b['nome']}\033[0m")
        print(f" Volume útil ({b['vol_util']:.1f} m³) cobre apenas "
              f"{b['eficiencia']:.1f}% da demanda.")
        print(f" Autonomia estimada: {b['autonomia']:.2f} dias.")
        print(f" ➔ Acionar abastecimento emergencial imediatamente.")

    for b in altos:
        print(f"\033[93m ⚠ ALTO – {b['nome']}\033[0m")
        print(f" Eficiência de {b['eficiencia']:.1f}%. Monitorar diariamente.")
        print(f" ➔ Planejar reforço de abastecimento.")

    for b in moderados:
        print(f"\033[33m ⚡ MODERADO – {b['nome']}\033[0m")

```

```

print(f"    Eficiência de {b['eficiencia']:.1f}%.")
print(f"    → Verificar possibilidade de redução de perdas.")

if alta_perda:
    print("\n\033[96m  💧 ALTA PERDA NA TUBULAÇÃO (> 20 %):\033[0m")
    for b in alta_perda:
        print(f"    • {b['nome']}: {b['perda']:.1f}% de perda")
    print("    → Inspeccionar e reparar a rede de distribuição.")

# _____
# MAIN
# _____

def main() -> None:
    bairros = coletar_dados()
    imprimir_tabela(bairros)
    imprimir_alertas(bairros)

    resp = input("\nDeseja gerar os gráficos comparativos? (S/n): ").strip().lower()
    if resp != "n":
        gerar_graficos(bairros, salvar=True)

    print("\n Análise concluída. Obrigado por usar o sistema!\n")

if __name__ == "__main__":
    main()

```

Comparação entre as LLMs:

O do **ChatGPT** parece o mais simples e direto, feito mais para funcionar rápido e ser fácil de entender.

O do **Gemini** já parece um pouco mais organizado e “arrumado”, com uma estrutura melhor e gráficos mais trabalhados.

O do **Claude** claramente é o mais completo e elaborado, com cara de projeto mais profissional e cheio de detalhes extras.

Critério	ChatGPT	Gemini	Claude
Tamanho do código	Médio	Médio	Muito grande
Complexidade	Baixa	Média	Alta
Facilidade de entender	Fácil	Muito fácil	Difícil
Organização	Básica	Boa	Excelente
Qualidade dos gráficos	Simples	Boa	Profissional

O código do ChatGPT é o mais simples, fácil de entender e possui gráficos básicos. O Gemini é mais organizado e apresenta gráficos melhores, mantendo uma complexidade intermediária. Já o Claude é o mais completo e profissional, com alta organização, muitos recursos e gráficos avançados, porém é também o mais complexo e difícil para iniciantes.